

Miguel Jaque Barbero - Curso 2025/26 - DAM/DAW

Introducción al Desarrollo de Software



Índice

Introducción.....	2
El Ordenador y la Arquitectura de Von Neumann.....	2
Lenguajes de Programación.....	2
Ingeniería de Software.....	3
Herramientas de Desarrollo.....	4
Conclusión.....	4

Introducción

El desarrollo de software no se limita a escribir código. Implica comprender el ecosistema completo que rodea a las aplicaciones informáticas: desde el hardware hasta las metodologías de trabajo y las herramientas que facilitan la tarea.

En estos apuntes veremos:

1. El ordenador y la arquitectura de Von Neumann.
2. Los lenguajes de programación.
3. La ingeniería de software y sus metodologías.
4. Las herramientas de desarrollo más utilizadas.

El Ordenador y la Arquitectura de Von Neumann

Un ordenador es una máquina que procesa datos, recibiendo información de entrada y generando información de salida.

En 1945, John Von Neumann definió una arquitectura que todavía hoy sigue siendo la base de los ordenadores modernos.

Los elementos principales son:

- Unidad Central de Proceso (CPU): interpreta y ejecuta instrucciones mediante la unidad de control, los registros y la ALU (unidad aritmético-lógica).
- Memoria principal: almacena tanto datos como instrucciones.
- Periféricos de entrada y salida: teclado, ratón, pantalla, impresora, disco duro, tarjeta de red, etc.
- Buses de comunicación: transportan datos, direcciones y señales de control entre los componentes.

Lenguajes de Programación

Un programa es un algoritmo escrito en un lenguaje de programación. Un algoritmo es una secuencia finita, clara y eficaz de instrucciones para resolver un problema.

Proceso de traducción de programas:

- Código fuente → compilador → código objeto.

- Código objeto + linkado → código ejecutable (código máquina).

En algunos lenguajes (Java, Python), el código se interpreta mediante una máquina virtual o un intérprete.

Clasificación histórica por generaciones:

1ª Generación: código máquina (unos y ceros).

2ª Generación: ensamblador (códigos mnemotécnicos).

3ª Generación: lenguajes de alto nivel (C, C++, Java, Python).

4ª Generación: lenguajes de propósito específico (SQL, MATLAB, HTML).

5ª Generación: basados en restricciones o lógicos (Prolog, Lisp).

Otros criterios de clasificación: paradigmas (imperativo, declarativo, orientado a objetos, funcional), compilados o interpretados, dependientes o independientes del sistema operativo, y de propósito general o específico.

Ingeniería de Software

La ingeniería de software aplica técnicas de ingeniería a la creación de programas, siguiendo un ciclo de vida bien definido.

Fases del ciclo de vida del software:

1. Estudio de viabilidad (técnica y económica).
2. Análisis de requisitos.
3. Diseño de la solución.
4. Construcción (programación).
5. Pruebas.
6. Implantación.
7. Mantenimiento.

Metodologías principales:

- Cascada: lineal y secuencial, clara pero poco flexible a cambios.
- Espiral: iterativa, incorpora retroalimentación frecuente.

- Programación extrema (XP): buenas prácticas al máximo nivel (pair programming, refactorización, test unitarios automáticos).
- Ágiles (Scrum): trabajo en iteraciones cortas (sprints), con roles definidos (Product Owner, Scrum Master, equipo de desarrollo).

Herramientas de Desarrollo

Cada fase del ciclo de vida cuenta con herramientas específicas:

- Análisis: casos de uso, historias de usuario, tableros Kanban (Trello, Jira).
- Diseño: UML, diagramas entidad-relación (MySQL Workbench), prototipos de interfaces (Figma).
- Construcción: IDEs (Eclipse, IntelliJ, Visual Studio), compiladores, frameworks (Django, Spring Boot), control de versiones (Git).
- Pruebas: JUnit, Postman, Selenium, SonarQube.
- Implantación: servidores (Apache, Nginx), contenedores (Docker), cloud (AWS, Azure).
- Mantenimiento: wikis colaborativas (MediaWiki, Confluence), sistemas de gestión de incidencias y comunicación (Slack, Google Chat).

Conclusión

El desarrollo de software es un proceso complejo que abarca:

- Conocimiento del hardware y la arquitectura de Von Neumann.
- Uso adecuado de lenguajes de programación según su propósito.
- Aplicación de metodologías de ingeniería de software que guíen el ciclo de vida.
- Empleo de herramientas que optimicen cada fase.

El objetivo final es construir aplicaciones eficaces, mantenibles y adaptables a los cambios.